

Implementačná dokumentácia k 1. úlohe do IPP 2024/2025

Meno a priezvisko: **Matúš Csirik**

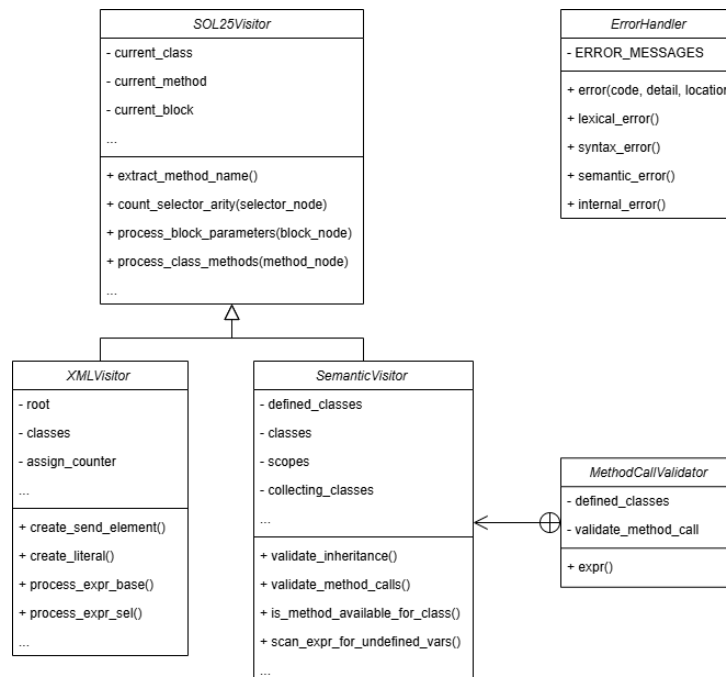
Login: **xcsirim00**

1. Opis riešenia

Implementovaný analyzátor slúži na spracovanie zdrojového kódu v jazyku *SOL25*, zabezpečuje lexikálnu, syntaktickú a sémantickú analýzu, ktorej výsledkom je štandardizovaná XML reprezentácia abstraktného syntaktického stromu (AST). Implementácia využíva Python 3.11 spolu s knižnicou *Lark* pre efektívnu syntaktickú analýzu.

Riešenie je postavené na objektovo orientovanej architektúre s dôrazom na nasledujúce princípy:

- **Dedičnosť:** Základným stavebným kameňom je trieda *SOL25Visitor*, ktorá definuje rozhranie pre prechádzanie AST. Táto trieda je rozšírená v triedach *SemanticVisitor* a *XMLVisitor*, ktoré implementujú špecifickú funkčnosť sémantickej validácie a generovania výsledného XML.
- **Návštevnícky vzor (Visitor):** Tento vzor umožňuje oddelenie dátovej štruktúry stromu od logiky jej spracovania, čím zvyšuje modularitu a uľahčuje rozširovanie funkcionality bez zmeny štruktúry AST.
- **Centralizovaná správa chýb:** Trieda *ErrorHandler* zabezpečuje jednotné spracovanie všetkých typov chýb, konzistentné chybové hlásenia a ukončenie programu pri detekcii problémov v zdrojovom kóde.



Obr. 1: Triedy a ich vzájomné vzťahy v projekte. *SOL25Visitor* generalizuje *XMLVisitor* a *SemanticVisitor*. *MethodCallValidator* je vnorená trieda. Parametre a metódy sú ilustračné.

2. Lexikálna a syntaktická analýza

Parser implementuje LALR(1) gramatiku definovanú v EBNF (Rozvinutá Backusova–Naurova forma) notácii prostredníctvom knižnice *Lark*. Pred samotným parsovaním sa vykonáva funkcia `validate_escape_sequences()`, ktorá overuje lexikálnu korektnosť reťazcov a deteguje problémy ako neukončené reťazce či neplatné escape sekvencie. Po úspešnej syntaktickej analýze je vygenerovaný AST, ktorý reprezentuje hierarchickú štruktúru programu v jazyku *SOL25*.

3. Sémantická analýza

Sémantická analýza v triede `SemanticVisitor` prebieha v dvoch fázach:

1. Zhromažďovanie symbolov:

- Spracovanie tried, ich dedičnosti a metód do interných štruktúr.
- Registrácia blokov a ich parametrov.
- Vytváranie tabuliek symbolov pre následné overenie.

2. Validácia:

- Overenie existencie všetkých referencovaných tried.
- Kontrola dedičnostnej hierarchie vrátane eliminácie cyklov.
- Kontrola triedy `Main` a metódy `run`.
- Validácia volania metód a správnosti ich parametrov.
- Overenie korektnej deklarácie a použitia premenných.

Na validáciu volaní metód je implementovaná vnorená trieda `MethodCallValidator`, ktorá prostredníctvom dedenia návštevníckeho vzoru precízne overuje, či volané metódy existujú v danej triede alebo jej predkoch. Riešenie prostredníctvom vnorenej triedy dovoľuje oddelenie tohoto procesu od zvyšku analýzy, čo umožnilo modulárny vývoj a uľahčilo testovanie.

4. Generovanie XML výstupu

XML reprezentácia je generovaná triedou `XMLVisitor`, ktorá rekurzívne prechádza AST a konštruuje zodpovedajúce elementy. Implementácia využíva knižnicu `xml.etree.ElementTree` a zabezpečuje:

- Generovanie elementov `<class>` s atribútmi názvu a rodičovskej triedy.
- Vytvorenie elementov `<method>` so správnymi selektormi.
- Spracovanie blokov kódu vrátane ich parametrov.
- Reprezentáciu priradení s príslušnými zdrojovými výrazmi.

Každý uzol stromu má samostatnú metódu spracovania. Vnorené štruktúry a výrazy sú generované rekurzívne.

5. Záver

Implementovaný parser predstavuje komplexné riešenie spracovania jazyka *SOL25* s dôrazom na modulárny a rozšriteľný objektovo orientovaný návrh. Generovaný XML výstup presne reflektuje štruktúru analyzovaného programu a je pripravený na následné spracovanie či interpretáciu. Cieľom riešenia bolo dosiahnuť funkčnú a udržiavateľnú implementáciu bez dôrazu na optimalizáciu výpočtovej náročnosti alebo pamäťovej efektivity.